

多模态项目代码解读

参考项目：<https://github.com/jingyaogong/minimind-v/tree/master>

其中LLM模块的介绍和代码：<https://github.com/jingyaogong/minimind/tree/master>

本代码旨在让大家理解多模态模型的训练流程，面试项目的描述参考 [居丽叶的简历项目5：K12数学题解答多模态模型全链路训练](#)。本代码为面试项目核心功能的最小化实现示例，与之对应的面试项目则是工业级的完整解决方案。鉴于工业级代码库涵盖了海量业务逻辑、工程化组件及运维模块，整体规模过于庞大，无法全面展示，故选取该最小化实现版本作为演示载体。

项目运行

1. 下载视觉模型和语言模型到指定目录下

代码块

```
1  # 下载clip模型到 ./model/vision_model 目录下
2  cd model/vision_model
3  git clone https://huggingface.co/openai/clip-vit-base-patch16
4  # 或者用modelscope下载
5  git clone https://www.modelscope.cn/models/openai-mirror/clip-vit-base-patch16
6  # 下载minimind语言模型权重到 ./out 目录下（作为训练VLM的基座语言模型）
7  cd ../../out
8  wget https://huggingface.co/jingyaogong/MiniMind2-V-PyTorch/resolve/main/llm_512.pth -O llm_512.pth
9  # 或者用modelscope下载
10 wget https://modelscope.cn/models/gongjy/MiniMind2-V-PyTorch/resolve/master/llm_512.pth -O llm_512.pth
```

2. 安装相关依赖

代码块

```
1  conda create -n mind python=3.12
2  conda activate mind
3  cd ../
4  pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3. 下载pretrain阶段的图片和文本数据

代码块

```
1  cd dataset/
2  wget https://hf-mirror.com/datasets/jingyaogong/minimind-v_dataset/resolve/main/pretrain_i2t.parquet
3  wget https://hf-mirror.com/datasets/jingyaogong/minimind-v_dataset/resolve/main/sft_i2t.parquet
```

4. **pretrain**：LLM加载MindMind模型pretrain的模型参数，视觉加载clip模型。训练后得到 `pretrain_vlm_*.pth` pretrain模型权重（其中*为模型的dimension，默认为512）。pretrain默认采用 `pretrain_data` 数据集，随机采样500条数据进行训练。

代码块

```
1  cd trainer/
2  python train_pretrain_vlm.py
```

5. **SFT**：加载pretrain的模型参数。训练后得到 `sft_vlm_*.pth` 作为指令微调模型权重。SFT阶段训练全部参数。SFT默认采用 `sft_data` 数据集，随机采样300条数据进行训练。

代码块

```
1 python train_sft_vlm.py
```

6. **RL**：加载SFT训练的模型参数，训练得到 `grpo_vlm_*.pth`。RL阶段训练全部参数。RL阶段默认采用 `sft_data` 数据集，随机采样100条数据进行训练。

代码块

```
1 python train_grpo_vlm.py
```

7. 另外还实现了一套用数学题目跑完整的pretrain、SFT、RL的流程，运行如下：

代码块

```
1 python train_pretrain_vlm_mathvista.py
2 python train_sft_vlm_mathvista.py
3 python train_grpo_vlm_mathvista.py
```

8. 模型测试

代码块

```
1 cd ../scripts/
2 python web_demo_vlm.py
```



Hi, I'm MiniMind2-V



关键代码解读

以下代码解读针对图像理解任务，解决数学题目的代码与前者类似，不做赘述。

pretrain

1. `train_epoch`：这里没有用 `transformers.Trainer` 之类的封装较好的库，而是自己实现了训练的流程。

```
train_grpo_vlm_mathvista.py train_grpo_vlm.py train_pretrain_vlm.py M X trainer_utils.py M eval
trainer > train_pretrain_vlm.py > train_epoch
19 from model.model_vlm import MiniMindVLM, VLMConfig
20 from dataset.lm_dataset import VLMDataset
21 from trainer.trainer_utils import get_lr, Logger, is_main_process, init_distributed_mode, s
22
23 warnings.filterwarnings('ignore')
24
25
26 def train_epoch(epoch, loader, iters, start_step=0, wandb=None):
27     """
28     训练流程:
29     1. 加载一个 batch 的数据 (文本 token + 图像像素)
30     2. 动态调整学习率 (余弦退火等策略)
31     3. 前向传播计算 logits
32     4. 计算交叉熵损失 (带 loss_mask 选择性计算)
33     5. 混合精度反向传播
34     6. 梯度累积 + 梯度裁剪 + 参数更新
35     7. 日志记录和模型保存
36     =====
37     """
38     loss_fct = nn.CrossEntropyLoss(reduction='none')
39     start_time = time.time()
```

a. 加载一个 batch 的数据 (文本 token + 图像), 动态调整学习率

```
代码块
1 for step, (X, Y, loss_mask, pixel_values) in enumerate(loader, start=start_step + 1):
2     X = X.to(args.device)
3     Y = Y.to(args.device)
4     loss_mask = loss_mask.to(args.device)
5     pixel_values = pixel_values.to(args.device)
6
7     # 动态学习率调整, 根据全局训练步数计算当前学习率, 全局步数 = epoch * 每个epoch的步数 + 当前步数
8     lr = get_lr(epoch * iters + step, args.epochs * iters, args.learning_rate)
9     for param_group in optimizer.param_groups:
10         param_group['lr'] = lr
```

b. 前向传播计算 logits, 带 loss_mask 计算交叉熵损失

```
代码块
1 with autocast_ctx:
2     # ==== 模型前向传播 ====
3     # 输入文本 token 和图像像素, 模型内部会:
4     # 1. 用 vision_encoder 编码图像 -> 图像特征
5     # 2. 用 vision_proj 将图像特征投影到 LLM 维度
6     # 3. 将图像特征嵌入到文本序列的对应位置
7     # 4. LLM 处理融合后的序列
8     res = model(X, pixel_values=pixel_values)
9     loss = loss_fct(
10         res.logits.view(-1, res.logits.size(-1)),
11         Y.view(-1)
12     ).view(Y.size())
13     # 只计算 mask=1 位置的平均损失
14     loss = (loss * loss_mask).sum() / loss_mask.sum()
15     # ==== 添加 MoE 辅助损失 ====
16     # 如果使用 MoE 架构, aux_loss 用于负载均衡 (防止部分专家过载)
17     loss += res.aux_loss
18     # ==== 梯度累积: 将损失除以累积步数 ====
19     loss = loss / args.accumulation_steps
```

c. 以混合精度进行反向传播, 采用梯度累积 + 梯度裁剪

```
代码块
```

```

1  # scaler.scale() 会将损失乘以一个缩放因子, 防止 float16 梯度下溢
2  scaler.scale(loss).backward()
3  if (step + 1) % args.accumulation_steps == 0:
4      # 将之前 scale 的梯度除以缩放因子, 恢复真实梯度值
5      scaler.unscale_(optimizer)
6      # 防止梯度爆炸, 将梯度范数限制在 grad_clip 以内
7      torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
8      scaler.step(optimizer)
9      scaler.update()
10     # 清零梯度
11     optimizer.zero_grad(set_to_none=True)

```

d. 日志记录和模型保存

代码块

```

1  if (step + 1) % args.accumulation_steps == 0:
2      # 将之前 scale 的梯度除以缩放因子, 恢复真实梯度值
3      scaler.unscale_(optimizer)
4      # 防止梯度爆炸, 将梯度范数限制在 grad_clip 以内
5      torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
6      scaler.step(optimizer)
7      scaler.update()
8      # 清零梯度
9      optimizer.zero_grad(set_to_none=True)
10
11  if step % args.log_interval == 0 or step == iters - 1:
12      spend_time = time.time() - start_time
13      # 恢复真实损失值 (之前被 accumulation_steps 除过)
14      current_loss = loss.item() * args.accumulation_steps
15      current_lr = optimizer.param_groups[-1]['lr']
16      eta_min = spend_time / (step + 1) * iters // 60 - spend_time // 60
17      Logger(f'Epoch:[{epoch+1}/{args.epochs}][{step}/{iters}] loss:{current_loss:.6f} lr:
{current_lr:.12f} epoch_Time:{eta_min}min:')
18      if wandb: wandb.log({"loss": current_loss, "lr": current_lr, "epoch_Time": eta_min})
19  if (step % args.save_interval == 0 or step == iters - 1) and is_main_process():
20      model.eval()
21      # ==== 保存模型权重和完整检查点 (包含优化器状态等, 用于断点续训) ====
22      vlm_checkpoint(vlm_config, weight=args.save_weight, model=model, optimizer=optimizer,
23                    epoch=epoch, step=step, wandb=wandb, save_dir=args.save_dir, scaler=scaler)
24      model.train()
25  del X, Y, loss_mask, pixel_values, res, loss
26  torch.cuda.empty_cache() # 清理 CUDA 缓存, 防止显存碎片

```

2. main 函数

```
train_pretrain_vlm.py M × trainer_utils.py M train_sft_vlm_mathvista.py train_pretrain_vlm_mathvista.py
trainer > train_pretrain_vlm.py > ...
26 def train_epoch(epoch, loader, iters, start_step=0, wandb=None):
103     del X, Y, loss_mask, pixel_values, res, loss
104     torch.cuda.empty_cache() # 清理 CUDA 缓存, 防止显存碎片
105
106
107 if __name__ == "__main__":
108     # 第 1 部分: 命令行参数解析 You, 现在 • Uncommitted changes
109     parser = argparse.ArgumentParser(description="MiniMind-V Pretrain")
110
111     # ----- 保存相关参数 -----
112     parser.add_argument("--save_dir", type=str, default="../out",
113                         help="模型保存目录")
114     parser.add_argument('--save_weight', default='pretrain_vlm', type=str,
115                         help="保存权重的前缀名, 最终文件名为 {save_weight}_{hidden_size}.pth")
116
117     # ----- 训练超参数 -----
118     parser.add_argument("--epochs", type=int, default=4,
119                         help="训练轮数 (epoch)")
120     parser.add_argument("--batch_size", type=int, default=16,
121                         help="每个设备的 batch size")
122     parser.add_argument("--learning_rate", type=float, default=4e-4,
123                         help="初始学习率, 会通过余弦退火动态调整")
124
125     # ----- 设备和精度 -----
126     parser.add_argument("--device", type=str, default="cuda:0" if torch.cuda.is_available() else "cpu",
127                         help="训练设备 (cuda:0/cuda:1/cpu)")
128     parser.add_argument("--dtype", type=str, default="bfloat16",
129                         help="混合精度类型 (bfloat16/float16) bfloat16 数值更稳定")
130
```

- a. 解析命令行参数
- b. 初始化分布式环境和随机种子（单卡环境可忽视）
- c. 配置目录、模型参数、检查点。如果启用断点续训，尝试加载检查点数据。保存和加载模型的代码见 `vlm_checkpoint`。

```
train_pretrain_vlm.py M model_vlm.py M trainer_utils.py M × train_sft_vlm_mathvista.py train_pretrain_vlm_mathvista.py
trainer > trainer_utils.py > vlm_checkpoint
81
82
83 def vlm_checkpoint(vlm_config, weight='pretrain_vlm', model=None, optimizer=None, epoch=0,
84                  os.makedirs(save_dir, exist_ok=True)
85                  moe_path = '_moe' if vlm_config.use_moe else ''
86                  ckp_path = f'{save_dir}/{weight}_{vlm_config.hidden_size}{moe_path}.pth'
87                  resume_path = f'{save_dir}/{weight}_{vlm_config.hidden_size}{moe_path}_resume.pth'
88
89 if model is not None: # 保存模式
90     from torch.nn.parallel import DistributedDataParallel
91     state_dict = model.module.state_dict() if isinstance(model, DistributedDataParallel) else model.state_dict()
92
93     # 保存模型权重 (half 精度, 用于推理/部署)
94     ckp_tmp = ckp_path + '.tmp'
95     torch.save({k: v.half().cpu() for k, v in state_dict.items()}, ckp_tmp)
96     os.replace(ckp_tmp, ckp_path)
97
98 # 获取 wandb/swanlab 运行 ID (用于续训时恢复日志)
99 wandb_id = None
100 if wandb:
101     if hasattr(wandb, 'get_run'):
102         run = wandb.get_run()
103         wandb_id = getattr(run, 'id', None) if run else None
104     else:
105         wandb_id = getattr(wandb, 'id', None)
```

- d. 设置混合精度训练
- e. 配置wandb训练监控
- f. 初始化模型、数据集、优化器。预训练阶段训练所有参数。模型初始化代码见 `init_vlm_model`。

```
train_pretrain_vlm.py M  model_vlm.py M  trainer_utils.py M X  train_sft_vlm_mathvista.py  train_pre...
trainer >  trainer_utils.py >  init_vlm_model
48 torch.backends.cudnn.benchmark = False
49
50
51 def init_vlm_model(vlm_config, from_weight='pretrain_vlm', tokenizer_path='../model',
52 vision_model_path='../model/vision_model/clip-vit-base-patch16',
53 save_dir='../out', device='cuda', freeze_llm=False):
54 """
55 freeze_llm=True 时: 只训练 vision_proj + vision_encoder, 冻结 llm
56 freeze_llm=False 时: 全参数训练 (vision_encoder + vision_proj + llm)
57 """
58 tokenizer = AutoTokenizer.from_pretrained(tokenizer_path)
59 model = MiniMindVLM(vlm_config, vision_model_path=vision_model_path)
60
61 if from_weight != 'none':
62     moe_suffix = '_moe' if vlm_config.use_moe else ''
63     weight_path = f'{save_dir}/{from_weight}_{vlm_config.hidden_size}{moe_suffix}.pth'
64     weights = torch.load(weight_path, map_location=device)
65     # strict=False: 允许权重文件中缺少部分参数 (如旧版本不含 vision_encoder)
66     model.load_state_dict(weights, strict=False)
67     Logger(f'已加载权重: {weight_path}')
68
69 # Pretrain阶段: 冻结 llm, 只训练 vision_encoder + vision_proj
70 if freeze_llm:
71     for name, param in model.named_parameters():
72         # 只训练 vision 相关参数
73         if 'vision_encoder' in name or 'vision_proj' in name:
74             param.requires_grad = True
75         else:
76             param.requires_grad = False
77
```

初始化数据集关注 `_generate_loss_mask` 函数，只对 `<|im_start|>assistant` 和 `<|im_end|>` 之间的内容计算损失。

```
train_pretrain_vlm.py  model_vlm.py  trainer_utils.py  train_sft_vlm.py  lm_dataset.py X
dataset >  lm_dataset.py
22 class VLMDataset(Dataset):
101 def _generate_loss_mask(self, input_ids):
102     """
103     生成损失掩码，用于只计算模型回复部分的损失（忽略用户输入部分）
104
105     策略：只对 <|im_start|>assistant 和 <|im_end|> 之间的内容计算损失
106     这样可以避免模型学习预测用户输入，只学习生成正确的回复
107
108     Args:
109     |   input_ids: tokenize 后的输入序列 (token ID 列表)
110
111     Returns:
112     |   loss_mask: 与 input_ids 等长的列表, 1 表示需要计算损失, 0 表示忽略
113     """
114     # 初始化掩码，默认全部为 0（不计算损失）
115     loss_mask = [0] * len(input_ids)
116     i = 0
117     while i < len(input_ids):
```

- g. 从检查点恢复状态
- h. 并行训练设置
- i. 调用 `train_epoch` 函数进行训练

SFT

1. `train_epoch`：与pretrain的代码结构类似，区别在于训练全量参数。


```
train_pretrain_vlm.py  model_vlm.py  lm_dataset.py  train_sft_vlm.py X  train_grpo_vlm.py

trainer > train_sft_vlm.py

20 from dataset.lm_dataset import VLMDataset
21 from trainer.trainer_utils import get_lr, Logger, is_main_process, init_distributed_mode,
22
23 warnings.filterwarnings('ignore')
24
25
26 def train_epoch(epoch, loader, iters, start_step=0, wandb=None):
27     loss_fct = nn.CrossEntropyLoss(reduction='none')
28     start_time = time.time()
29     for step, (X, Y, loss_mask, pixel_values) in enumerate(loader, start=start_step + 1):
30         """
31         数据说明:
32         - X: 输入 token 序列 [batch_size, seq_len - 1] (去掉最后一个 token)
33         - Y: 目标 token 序列 [batch_size, seq_len - 1] (去掉第一个 token, 用于预测下一个 token)
34         - loss_mask: 损失掩码, 只对 assistant 回复部分计算损失, 1 表示计算, 0 表示忽略
35         - pixel_values: 图像张量 [batch_size, num_images, channels, height, width]
36         """
37         X = X.to(args.device)
38         Y = Y.to(args.device)
39         loss_mask = loss_mask.to(args.device)
```

2. main 函数

```
train_vlm.py M  model_vlm.py M  lm_dataset.py M  train_sft_vlm.py M X  tra

trainer > train_sft_vlm.py > ...

95
96
97 if __name__ == "__main__":
98     # 第 1 部分: 命令行参数解析
99     parser = argparse.ArgumentParser(description="MiniMind-V SFT")
100
101     # ----- 保存相关参数 -----
102     parser.add_argument("--save_dir", type=str, default="../out", help=
103     parser.add_argument('--save_weight', default='sft_vlm', type=str, h
104
105     # ----- 训练超参数 ----- You, 前天 • Uncommitted chang
106     parser.add_argument("--epochs", type=int, default=2, help="训练轮数"
107     parser.add_argument("--batch_size", type=int, default=4, help="每个i
108     parser.add_argument("--learning_rate", type=float, default=1e-6, he
109     parser.add_argument("--accumulation_steps", type=int, default=1, he
110     parser.add_argument("--grad_clip", type=float, default=1.0, help="梯
111
112     # ----- 设备和精度 -----
113     parser.add_argument("--device", type=str, default="cuda:0" if torch
114     parser.add_argument("--dtype", type=str, default="bfloat16", help="
115     parser.add_argument("--num_workers", type=int, default=8, help="数据
116
117     # ----- 日志和保存间隔 -----
118     parser.add_argument("--log_interval", type=int, default=100, help="
```

- a. 解析命令行参数
- b. 初始化分布式环境和随机种子（单卡环境可忽视）
- c. 配置目录、模型参数、检查点。如果启用断点续训，尝试加载检查点数据。保存和加载模型的代码见 `vlm_checkpoint`。
- d. 设置混合精度训练
- e. 配置wandb训练监控
- f. 初始化模型、数据集、优化器。SFT阶段训练所有参数。模型初始化代码见 `init_vlm_model`。
- g. 从检查点恢复状态
- h. 并行训练设置
- i. 调用 `train_epoch` 函数进行训练

RL

- 1. `compute_reward` 函数：GRPO的奖励函数，从以下3个角度计算奖励（经pretrain和SFT两个阶段，模型已经有比较强的图像描述能力，因此没有设置正确性奖励，而是只从描述的丰富度、流畅性、格式方式入手）：
 - 格式与长度奖励: 鼓励输出格式规范（正确标点、无乱码）且长度适中的描述，确保生成内容完整可读

- 词汇丰富度奖励: 鼓励使用多样化的词汇和描述性表达，避免单调重复的用词
- 流畅性奖励: 惩罚连续重复词和重复短语，确保生成的文本自然流畅

```
model_vlm.py M lm_dataset.py M trainer_utils.py M train_grpo_vlm.py X
trainer > train_grpo_vlm.py > compute_reward
148
149
150 def compute_reward(response: str) -> float:
151     """
152     计算图片描述任务的综合奖励
153
154     奖励组成 (每个奖励范围 0-1, 总分范围 0-4):
155     - 长度奖励: 0 ~ 1
156     - 词汇丰富度: 0 ~ 1
157     - 流畅性: 0 ~ 1
158     - 格式: 0 ~ 1
159     """
160     reward = 0.0
161     reward += length_reward(response)
162     reward += vocabulary_diversity_reward(response)
163     reward += fluency_reward(response)
164     reward += format_reward(response)
165     return reward
166
167
```

2. GRPOVLMDataset：只返回prompt，因为奖励计算时不计算正确性奖励，所以不需要response。

```
model_vlm.py M trainer_utils.py M train_grpo_vlm.py X train_sft_vlm.py M eval_vlm.py
trainer > train_grpo_vlm.py > GRPOVLMDataset
204 # ===== 数据集 =====
205 class GRPOVLMDataset(Dataset):
206     """
207     GRPO 训练数据集
208     与 SFT 数据集的区别：
209     - SFT: 返回完整的 (prompt + response), 用于监督学习
210     - GRPO: 只返回 prompt, 模型自己生成响应后计算奖励
211     """
212
213     def __init__(
214         self,
215         data_path: str,
216         images_path: str,
217         tokenizer,
218         preprocess,
219         image_special_token: str = '@' * 196,
220         max_length: int = 512,
221     ):
222         self.data_path = data_path
```

3. GRPOTrainer.train_step：执行一个 GRPO 训练step

```
trainer_utils.py M train_grpo_vlm.py X train_sft_vlm.py M train_grpo_vlm_mathvista.py
trainer > train_grpo_vlm.py > GRPOTrainer > train_step
300
301
302 # ===== GRPO Trainer =====
303 class GRPOTrainer(GRPOTrainerBase):
304     def train_step(self, batch):
305         """
306         执行一个 GRPO 训练步骤
307
308         GRPO 算法流程：
309         1. 对每个 prompt 生成 num_generations 个响应
310         2. 计算每个响应的奖励
311         3. 在组内计算相对优势 (advantage = (reward - mean) / std)
312         4. 计算策略损失
313         5. 计算 KL 惩罚项 (防止策略偏离参考模型太远)
314         6. 反向传播更新模型
315         """
```

- generate_responses：batch中的每个 prompt 生成 num_generations 个响应，将所有响应对齐到相同长度并解码成文本，返回生成的 token ids 和对应的文本。注意这里是在生成的响应的基础上，右边加padding，为了后续计算 log_probs 方便。


```

1 generated_ids, generated_attention_mask, response_texts = self.generate_responses(
2     prompt_ids, attention_mask, pixel_values
3 )

```

举个例子：

代码块

```

1 形状: [batch_size, prompt_len]
2
3 示例 (batch_size=2, prompt_len=5):
4  prompt1 原始: [A, B, C]      → 左填充后: [pad, pad, A, B, C]
5  prompt2 原始: [X, Y, Z, W, V] → 无需填充: [X, Y, Z, W, V]
6
7 attention_mask:
8  [[0, 0, 1, 1, 1], ← prompt1: 前2个是pad=0, 后3个是真实token=1
9   [1, 1, 1, 1, 1]] ← prompt2: 全是真实token=1
10 形状: [batch_size * num_generations, max_gen_len]
11
12 假设 num_generations=2, 生成后:
13  seq1: [pad, pad, A, B, C, R1, R2]      长度7
14  seq2: [pad, pad, A, B, C, R3]          长度6 (提前遇到EOS)
15  seq3: [X, Y, Z, W, V, R4, R5, R6]      长度8
16  seq4: [X, Y, Z, W, V, R7, R8]          长度7
17
18 右填充对齐到 max_len=8 后:
19  seq1: [pad, pad, A, B, C, R1, R2, pad]
20  seq2: [pad, pad, A, B, C, R3, pad, pad]
21  seq3: [X, Y, Z, W, V, R4, R5, R6]
22  seq4: [X, Y, Z, W, V, R7, R8, pad]
23
24 generated_attention_mask (4, 8):
25  [[0, 0, 1, 1, 1, 1, 1, 0], ← seq1: 左pad=0, 真实token=1, 右pad=0
26   [0, 0, 1, 1, 1, 1, 0, 0], ← seq2: 生成更短, 右边2个pad
27   [1, 1, 1, 1, 1, 1, 1, 1], ← seq3: 全是真实token
28   [1, 1, 1, 1, 1, 1, 1, 0]] ← seq4: 右边1个pad

```

```

trainer > trainer_utils.py > GRPOTrainerBase > __init__
231 class GRPOTrainerBase:
259     def generate_responses(self, prompt_ids, attention_mask, pixel_values):
260         """
261         为每个 prompt 生成 num_generations 个响应
262
263         Returns:
264         all_responses: [batch * num_generations, max_seq_len] 的 token ids
265         all_attention_masks: [batch * num_generations, max_seq_len] 的 attention mask
266         all_response_texts: 解码后的文本列表
267         """
268         batch_size = prompt_ids.size(0)
269         all_responses = []
270         all_attention_masks = []
271         all_response_texts = []
272
273         self.model.eval()
274
275         for _ in range(self.num_generations):
276             generated, gen_attention_mask = self._generate_single(prompt_ids, attention_ma

```

`_generate_single`：执行单次自回归生成，每次用模型预测下一个 token 的概率分布，按温度采样选择下一个 token，拼接到序列末尾，直到生成结束符或达到最大长度。

```
model_vlm.py M  trainer_utils.py M X  train_grpo_vlm.py  train_sft_vlm.py M  train_grpo_vlm_m
trainer > trainer_utils.py > GRPOTrainerBase > generate_responses
231 class GRPOTrainerBase:
299
300     def _generate_single(self, prompt_ids, attention_mask, pixel_values):
301         """自回归生成单个响应: 每次预测下一个 token, 直到 EOS 或达到最大长度"""
302         batch_size, prompt_len = prompt_ids.shape
303
304         generated = prompt_ids.clone()
305         current_attention_mask = attention_mask.clone()
306         finished = torch.zeros(batch_size, dtype=torch.bool, device=self.device)
307
308         for _ in range(self.max_new_tokens):
309             if finished.all():
310                 break
311
312             with torch.no_grad():
313                 outputs = self.model(
314                     input_ids=generated,
315                     attention_mask=current_attention_mask,
316                     pixel_values=pixel_values,
317                 )
318
```

b. 对生成的图像描述计算奖励

代码块

```
1 rewards = []
2 for response in response_texts:
3     reward = compute_reward(response)
4     rewards.append(reward)
5
6 rewards = torch.tensor(rewards, dtype=torch.float32, device=self.device)
```

c. 计算组内相对优势, $A_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}$

代码块

```
1 rewards_grouped = rewards.view(batch_size, self.num_generations)
2 mean_rewards = rewards_grouped.mean(dim=1, keepdim=True)
3 std_rewards = rewards_grouped.std(dim=1, keepdim=True) + 1e-8 # 防止除零
4 advantages = (rewards_grouped - mean_rewards) / std_rewards
5 advantages = advantages.view(-1).detach()
```

d. 设置mask矩阵, `prompt_mask` 用于区分 prompt 部分和生成部分, 只对生成部分计算损失

代码块

```
1 # 扩展 pixel_values 以匹配生成的响应数量
2 expanded_pixel_values = pixel_values.repeat_interleave(self.num_generations, dim=0)
3 # 使用生成时产生的 attention_mask, 正确标记 pad 位置
4 expanded_attention_mask = generated_attention_mask
5
6 # prompt_mask 用于区分 prompt 部分和生成部分, 只对生成部分计算损失
7 prompt_mask = torch.zeros_like(generated_ids)
8 prompt_mask[:, :prompt_len] = 1
```

e. `compute_log_probs`: 计算旧的策略模型和参考模型的 `log_probs`。将序列输入模型获取每个位置的 logits, 用 `log_softmax` 转为对数概率, 然后只对响应部分 (排除 prompt) 的 token 求平均对数概率, 用于后续的策略优化。
`old_token_log_probs` 和 `ref_avg_log_probs` 表示 $\log \pi_{\theta_{old}}(a|s)$ 和 $\log \pi_{ref}(a|s)$, s 是 prompt+image, a 是生成的 response。

代码块

```
1 with torch.no_grad():
```

```

2     _, old_token_log_probs, response_mask = self.compute_log_probs(
3         self.model, generated_ids, expanded_attention_mask,
4         expanded_pixel_values, generated_ids, prompt_mask
5     )
6
7     # 计算参考模型的 log probability (用于 KL 惩罚)
8     with torch.no_grad():
9         _, ref_token_log_probs, _ = self.compute_log_probs(
10             self.ref_model, generated_ids, expanded_attention_mask,
11             expanded_pixel_values, generated_ids, prompt_mask
12         )

```



举个例子：

代码块

```

1  示例：序列 [pad, pad, A, B, C, R1, R2, pad]
2      - 左边的 pad：输入 prompt 时的左填充（让 prompt 末尾对齐，便于生成）
3      - 右边的 pad：生成后的右填充（让完整序列长度对齐，便于批量计算）
4
5  prompt_mask      = [1, 1, 1, 1, 1, 0, 0, 0]  ← prompt部分（含左pad）=1
6  1 - prompt_mask  = [0, 0, 0, 0, 0, 1, 1, 1]  ← 非prompt部分
7  attention_mask   = [0, 0, 1, 1, 1, 1, 1, 0]  ← 真实token=1, pad=0
8  response_mask    = [0, 0, 0, 0, 0, 1, 1, 0]  ← 只有R1,R2保留
9  效果：只保留 response 的真实 token

```

f. 计算当前策略的 `log_probs`。

- TRL 和 veRL 采用采样与训练分离的异步架构：先用当前策略批量生成大量 responses 并保存采样时的 log_probs 到 old_per_token_logps，然后在后续多个训练步骤中复用这批数据，这种方式通常配合 vLLM 等高效推理引擎，实现推理和训练的流水线并行，吞吐量更高。
- 本代码中采用同步架构：每个 batch 采样后立即训练， π_{old} 是在 train_step 开始时用 torch.no_grad() 计算并冻结的当前模型的 log_probs，引入了 num_inner_updates 内循环，在同一个 batch 上多次更新 π_{θ} ，而 π_{old} 保持不变，这样 ratio 就会随着内循环迭代逐渐偏离 1，来模拟异步采样的效果。

代码块

```

1  num_inner_updates = 4
2  for _ in range(num_inner_updates):
3      # 计算  $\pi_{\theta}$  的 per-token log prob

```

```

4     _, token_log_probs, _ = self.compute_log_probs(
5         self.model, generated_ids, expanded_attention_mask,
6         expanded_pixel_values, generated_ids, prompt_mask
7     )

```

g. 计算重要性比率: $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)} = \exp(\log \pi_\theta - \log \pi_{ref})$

代码块

```

1  log_ratio = avg_log_probs - ref_avg_log_probs
2  ratio = torch.exp(log_ratio.clamp(-10, 10)) # 限制范围防止数值溢出

```

h. 计算损失: `policy_loss` 表示 $L^{CLIP}(\theta) = -\mathbb{E}[\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)]$, `kl_div` 表示 $D_{KL}(\pi_\theta || \pi_{ref}) \approx (r_t - 1) - \log r_t$

代码块

```

1  clipped_ratio = torch.clamp(ratio, 1.0 - self.clip_range, 1.0 + self.clip_range)
2  policy_loss1 = -advantages * ratio
3  policy_loss2 = -advantages * clipped_ratio
4  policy_loss = torch.max(policy_loss1, policy_loss2)

```

i. KL散度 $D_{KL}(\pi_\theta || \pi_{ref}) \approx (r_t - 1) - \log r_t$

代码块

```

1  per_token_kl = torch.zeros_like(per_token_loss)
2  if self.beta > 0:
3      ref_log_ratio = token_log_probs - ref_token_log_probs
4      ref_ratio = torch.exp(ref_log_ratio.clamp(-10, 10))
5      per_token_kl = (ref_ratio - 1) - ref_log_ratio

```

j. 最终损失函数: $L(\theta) = L^{CLIP}(\theta) + \beta \cdot D_{KL}(\pi_\theta || \pi_{ref})$ 。只计算回复部分的损失。

代码块

```

1  per_token_loss = per_token_loss + self.beta * per_token_kl
2
3  # ==== 计算 masked loss (只对 response 部分) ====
4  # loss = sum(per_token_loss * mask) / sum(mask)
5  masked_loss = (per_token_loss * response_mask).sum(dim=-1)
6  seq_lengths = response_mask.sum(dim=-1).clamp(min=1)
7  loss = (masked_loss / seq_lengths).mean()

```

k. 反向传播

代码块

```

1  self.optimizer.zero_grad()
2  loss.backward()
3  torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
4  self.optimizer.step()

```

l. 汇总指标

```
1     avg_ratio = ((ratio * response_mask).sum() / response_mask.sum().clamp(min=1)).item()
2     avg_kl = ((per_token_kl * response_mask).sum() / response_mask.sum().clamp(min=1)).item()
3
4     metrics = {
5         'loss': loss.item(),
6         'kl_div': avg_kl,
7         'mean_reward': rewards.mean().item(),
8         'max_reward': rewards.max().item(),
9         'min_reward': rewards.min().item(),
10        'ratio': avg_ratio,
11    }
12
13    return loss, metrics
```

4. main 函数



- a. 命令行参数解析
- b. 初始化环境，设置随机种子和路径
- c. 配置模型参数、检查点。
- d. 初始化策略模型和参考模型，RL阶段训练所有策略模型的参数。模型初始化代码见 `init_vlm_model`。
- e. 准备数据集
- f. 配置 wandb 日志
- g. 初始化优化器和 `GRPOTrainer`
- h. 调用 `train_step` 进行训练
- i. 保存模型

最后关注 `trainer_utils.py` 里的代码：

- 1. 将一个批次中的多个样本整理成统一格式的张量
 - a. 文本序列左填充（Left Padding）：将批次中不同长度的 prompt_ids 对齐到最大长度，采用左填充方式（保证推理时 prompt和response之间不会有pad），并生成对应的 attention_mask，确保模型不会关注pad位置。
 - b. 图像张量填充：处理批次中每个样本可能包含不同数量图像的情况，将 pixel_values 填充到统一的 [batch_size, max_images, C, H, W] 形状。

model_vlm.py M trainer_utils.py M × train_grpo_vlm.py train_pretrain_vlm.py

trainer > trainer_utils.py > ...

174

class GRPOCollator:

175

"""GRPO 数据整理器 (公共基类)"""

176

177

def __init__(self, pad_token_id=0):

178

| self.pad_token_id = pad_token_id

179

180

def __call__(self, features):

181

| max_len = max(f['prompt_ids'].size(0) for f in features)

182

183

| prompt_ids_list = []

184

| attention_mask_list = []

185

186

| for f in features:

187

| | prompt_ids = f['prompt_ids']

188

| | pad_len = max_len - prompt_ids.size(0)

189

2. GRPOTrainerBase 中的函数之前已经介绍过

LLM模型架构

model_minimind.py 代码

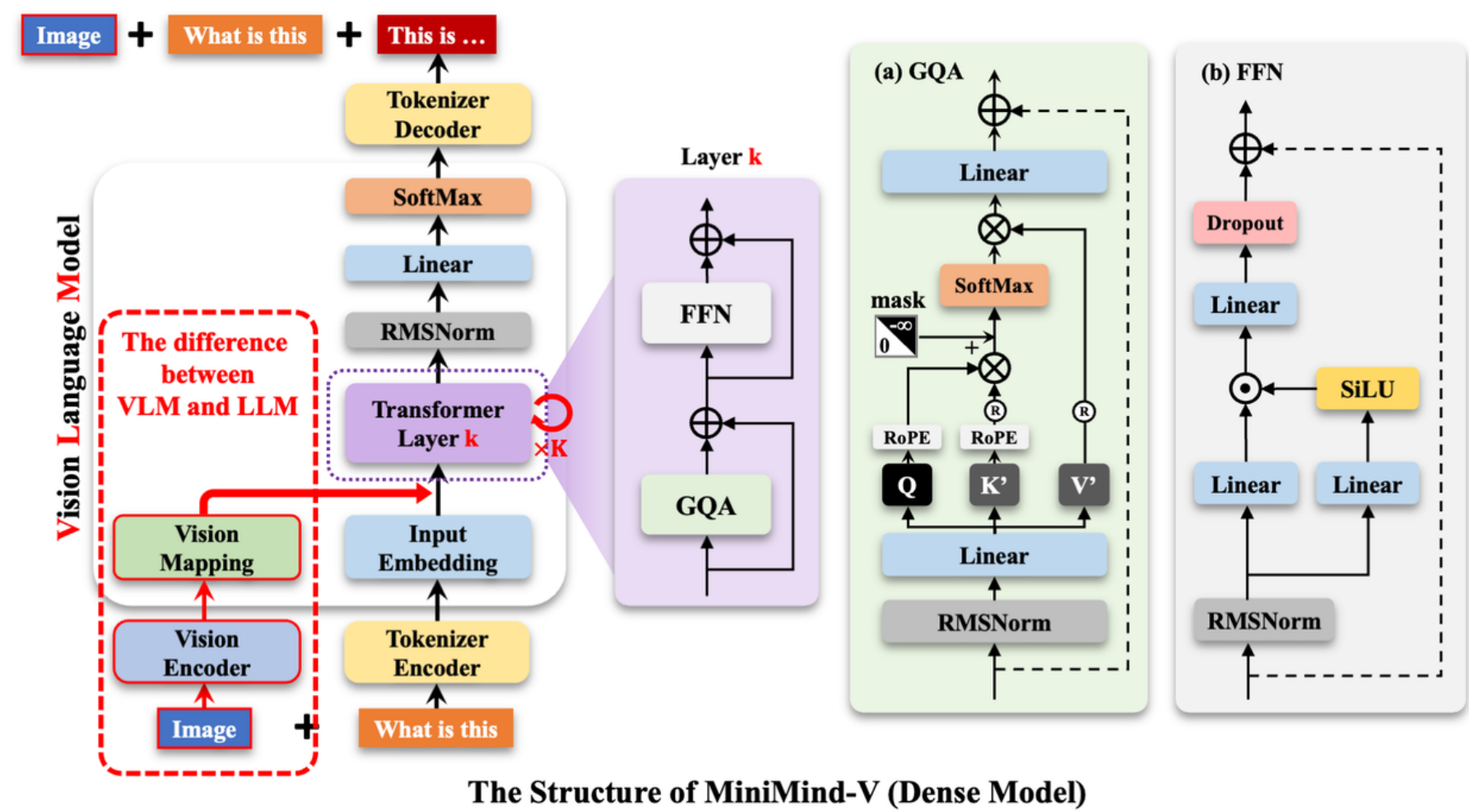
1. MiniMindConfig：配置模型架构参数

参数	默认值	说明
hidden_size	512	隐藏层维度
num_hidden_layers	8	Transformer 层数
num_attention_heads	8	注意力头数
num_key_value_heads	2	KV 头数（GQA 技术）
vocab_size	6400	词表大小
max_position_embeddings	32768	最大位置长度
rope_theta	1.00E+06	RoPE 基础频率
flash_attn	TRUE	是否使用 Flash Attention
use_moe	FALSE	是否启用混合专家
n_routed_experts	4	路由专家数量
num_experts_per_tok	2	每个 token 激活的专家数
n_shared_experts	1	共享专家数量
aux_loss_alpha	0.01	辅助损失权重（平衡专家负载）

2. RMSNorm：
$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma$$
3. precompute_freqs_cis：预先计算 RoPE 的 cos/sin 值，
$$\theta_i = \frac{1}{100000^{2i/d}}$$
4. apply_rotary_pos_emb：对 Q 和 K 应用旋转位置编码
5. repeat_kv：为了支持 GQA，做 KV 复制
6. Attention：注意力模块，采用GQA、RoPE、kv cache、flashattention。
7. FeedForward：前馈神经网络，采用SwiGLU 激活函数
8. MoEGate：门控网络，决定每个 token 由哪些专家处理，添加辅助损失，鼓励专家负载均衡。
9. MOEFeedForward：每个token都激活top-k 个专家，另外还有所有token都激活的共享专家。
10. MiniMindBlock：transformer block：输入 → RMSNorm → Attention → + 残差 → RMSNorm → FFN/MOE → + 残差 → 输出
11. MiniMindModel：Transformer Decoder 主干网络。

12. MiniMindForCausalLM：完整的因果语言模型，模型输出是下一个token的概率。

VLM模型架构

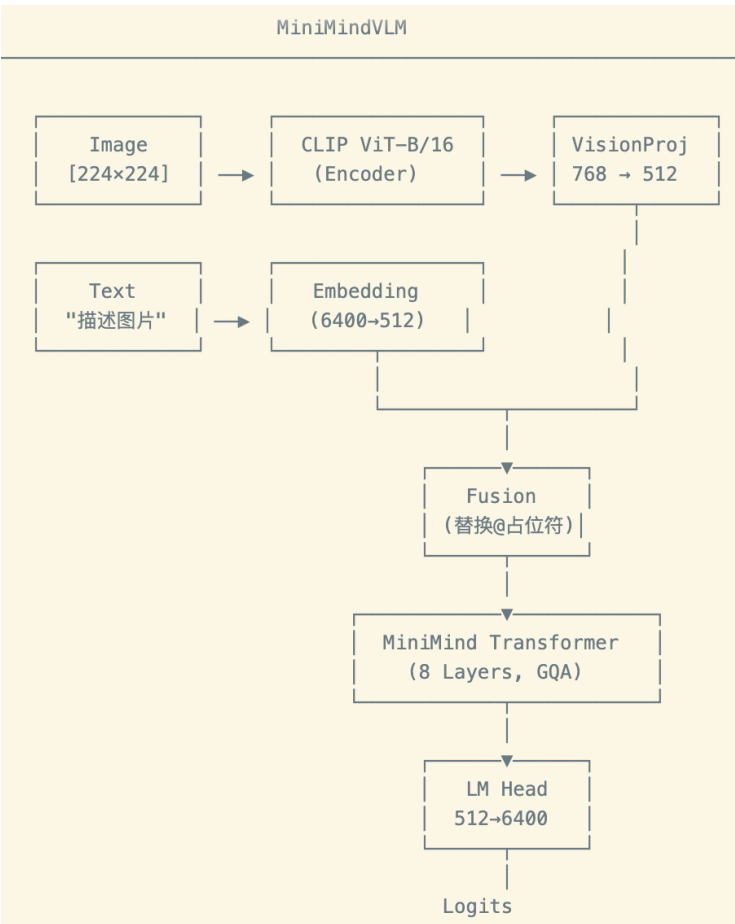


model_vlm.py 代码，在语言模型的基础上添加了视觉模块

1. VLMConfig：视觉模型相关参数配置，这里设置为196是因为CLIP ViT-Base/16 将 224×224 图像分割成 $14 \times 14 = 196$ 个 patch，每个 patch 对应一个视觉 token，所以需要 196 个占位符。

参数	默认值	说明
image_special_token	'@' * 196	图像占位符字符串（196 个 @ 字符）
image_ids	[34] * 196	图像占位符的 token ID 列表

2. VisionProj：将 CLIP 视觉编码器的特征维度投影到 LLM 的隐藏维度， $768 \rightarrow 512$
3. MiniMindVLM：



iii. 位置编码

代码块

```
1 position_embeddings = (  
2     self.model.freqs_cos[start_pos:start_pos + seq_length],  
3     self.model.freqs_sin[start_pos:start_pos + seq_length]  
4 )
```

iv. 前向传播

代码块

```
1 for layer_idx, (layer, past_key_value) in enumerate(zip(self.model.layers, past_key_values)):  
2     hidden_states, present = layer(  
3         hidden_states,  
4         position_embeddings,  
5         past_key_value=past_key_value,  
6         use_cache=use_cache,  
7         attention_mask=attention_mask  
8     )  
9     presents.append(present)
```

v. 输出 Logits

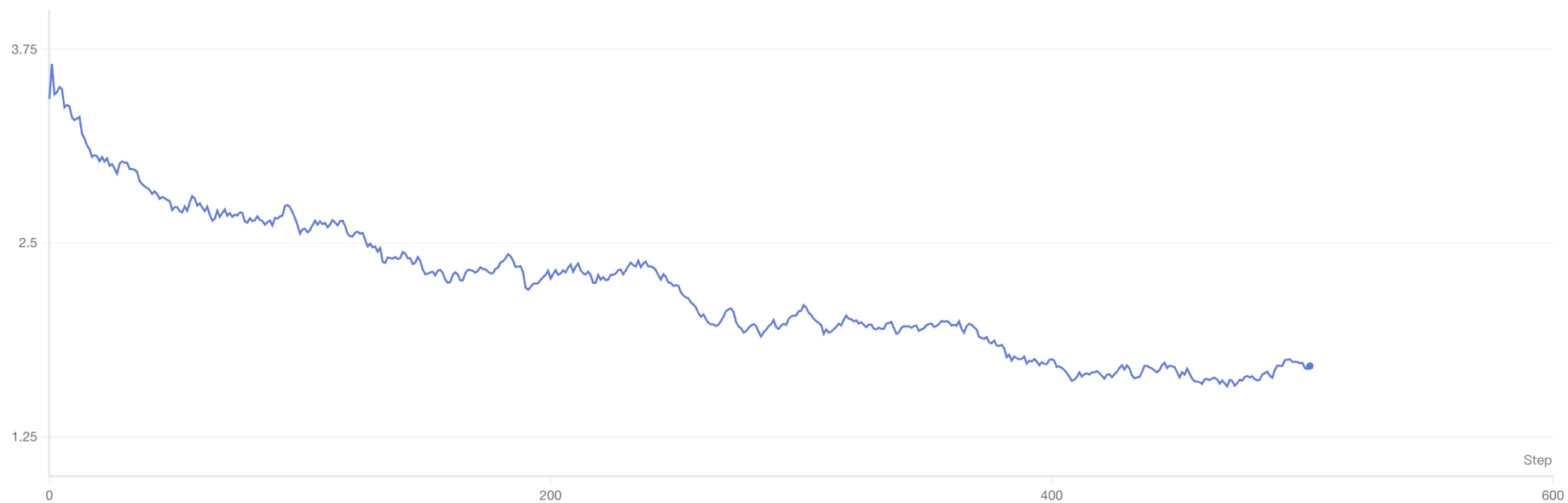
代码块

```
1 hidden_states = self.model.norm(hidden_states)  
2  
3     aux_loss = sum(  
4         layer.mlp.aux_loss  
5         for layer in self.model.layers  
6         if isinstance(layer.mlp, MOEFeedForward)  
7     )  
8     slice_indices = slice(-logits_to_keep, None) if isinstance(logits_to_keep, int) else  
logits_to_keep  
9     logits = self.lm_head(hidden_states[:, slice_indices, :])  
10    output = CausalLMOutputWithPast(logits=logits, past_key_values=presents,  
hidden_states=hidden_states)  
11    output.aux_loss = aux_loss  
12    return output
```

训练结果

Pretrain：在图文对齐数据[Chinese-LLaVA-Vision](#) 上进行训练，学习图片的通用知识，可以看到损失在随着训练明显下降。

loss



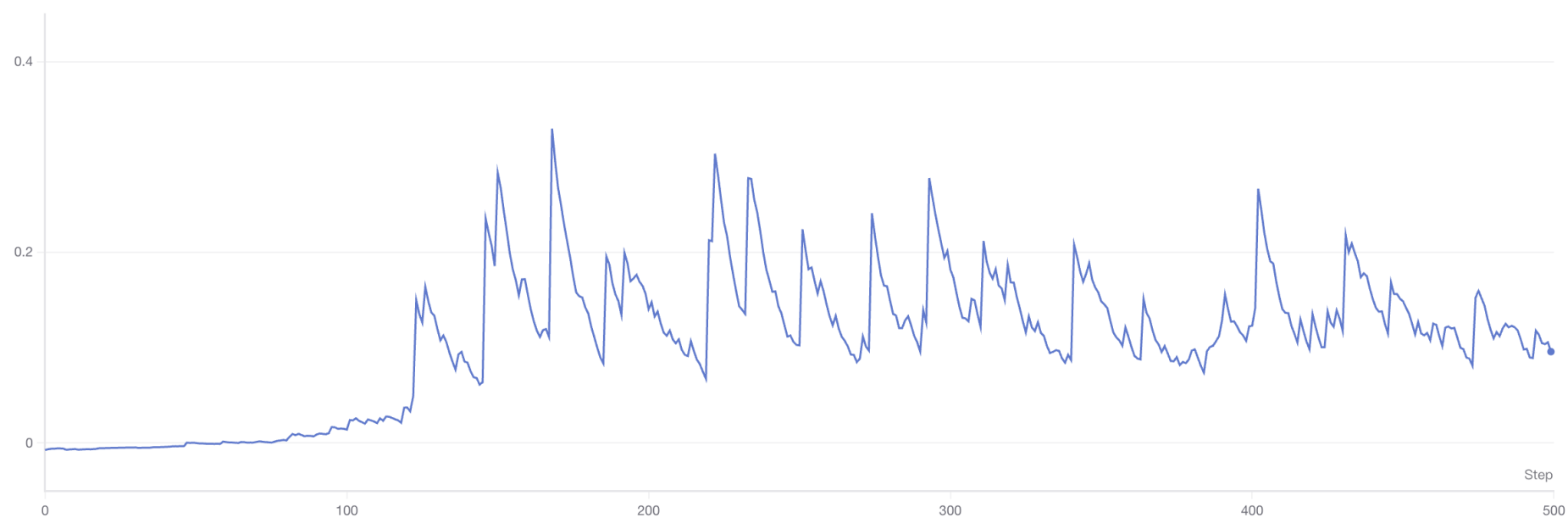
SFT：在指令遵循数据 [llava-en-zh-300k](#) 上训练，SFT阶段加长了上下文长度，损失也在明显下降。

loss



RL：仍然在指令遵循任务上训练，这里是GRPO的损失函数，与pretrain和sft阶段的交叉熵损失不同。损失增大主要来源于KL散度的增大。

loss



奖励在50step就稳定到了2.8左右，后面的训练并没有带来提升。

mean_reward

